RailRoute Lab

Time-aware A* shuttle routing on a fictional campus

REPORT STATUS

Draft for team verification before submission. This report describes the running repository and measured experiments. Every member should check the code, numbers, wording, and actual contribution record before submitting it.

PROBLEM AND MOTIVATION

Given a start location, destination, departure time, and possible railway-tunnel closures, find the drivable route with the smallest modeled in-vehicle travel time. Congestion changes by time of day, so the fastest path can change even when the start and goal do not. A railway barrier creates a useful routing constraint: the graph has only two crossing roads.

This is a controlled **search experiment**, not a deployed campus navigator. Fourteen named locations and seventeen roads form a reproducible synthetic graph. Coordinates, speed, congestion, and delay values are generated assumptions. No measured VIT road or timetable data are used.

WHAT IS IMPLEMENTED

- A local Python A* search whose priority is **f = g + h**, with an admissible straight-line travel-time heuristic.
- A Dijkstra comparison mode using the same search engine with **h = 0**.
- Arrival-time-dependent road costs, closure handling, route reconstruction, and a recorded search trace.
- A terminal demo and browser view that accept varied inputs and expose the algorithm's decisions.

TEAM

This is a five-person project. Add verified member names and actual contributions before final submission. The original proposal's planned task split is not treated as a record of who implemented the current code.

MAIN RESULT

On 23,520 reproducible input cases, A* and Dijkstra agreed on reachability and route cost. Across the 20,640 reachable cases, A* expanded 5.598 locations on average versus 7.012 for Dijkstra. This is a consistency result for a small synthetic model, not a real-world performance claim.

AI formulation

State, action, transition, goal, and cost are explicit

SEARCH PROBLEM

Component	Definition
State	Road-graph location together with arrival time
Initial state	Chosen start at the requested departure time
Action	Traverse a connected road that is open
Transition	Reach adjacent node; advance by entry-time road duration
Goal test	Current location equals selected destination
Path cost	Sum of modeled road travel seconds

The route objective is **minimum in-vehicle travel time**. Waiting for a shuttle, passenger demand, road legality outside the test graph, and schedule constraints are not included. The only blocked roads exposed as inputs are the west and east tunnel edges.

GRAPH AND TRANSITION MODEL

Let G=(V,E), with V the fourteen locations and E the seventeen bidirectional roads. For edge (u,v), the time cost c(u,v,t) depends on the clock time t at which the shuttle enters that road. If it arrives at u after elapsed time g(u), then its candidate arrival at v is **g(v)=g(u)+c(u,v, departure+g(u))**. Closed tunnel edges are skipped.

Each road has a free-flow traversal time. The model multiplies that time by a congestion factor determined by time of day. It integrates progress across period boundaries, so a road started at 09:59 may use both peak and off-peak rates. The resulting edge arrival function is FIFO: a later departure on the same road cannot arrive before an earlier one solely because a time bucket changed.

WHY ONE BEST ARRIVAL PER NODE SUFFICES

With FIFO edge travel and no waiting reward, reaching the same location earlier is never worse than reaching it later. The search therefore stores only the best known elapsed arrival g for each location. If future actions could benefit from waiting, reservations, capacity, or non-FIFO travel, the state representation would need to change.

SUCCESS AND FAILURE OUTPUTS

On success the system returns the route, total time, total distance, expansion trace, and comparison result. If both tunnels are closed for a cross-rail trip, the queue empties without reaching the goal; the terminal and browser demos report no route and retain the explored locations.

Model and heuristic

Assumptions that make the experiment reproducible

SYNTHETIC NETWORK

The generator in sample_map.py uses an integer seed to perturb coordinates, road distances, free-flow speeds, and congestion factors while keeping the same connected topology. Seed 17 is the default case. Nodes north and south of the railway connect only through two tunnel edges. This lets one input demonstrate a west route, an east route, or no crossing route.

The assumed maximum shuttle speed is 15 km/h. Every generated road length is at least its straight-line length, free-flow time respects the maximum speed, and congestion factors are at least 1. Morning peak is 07:00–10:00; evening peak is 16:00–19:00. These are scenario settings, not measured traffic periods.

HEURISTIC

For current location n and destination z, **h(n) = straight-line-distance(n,z) / 15 km/h**, converted to seconds. For example, a 0.50 km straight-line separation gives a 2.0 minute lower bound. The actual road path cannot be shorter than that line, and no modeled road travels faster than 15 km/h. Congestion only increases road time. Therefore h never exceeds the true remaining modeled travel time: it is **admissible**.

The triangle inequality plus the edge speed bound also gives $h(u) \leq c(u,v,t)+h(v)$ for every open road and entry time. This is **consistency**. Together with FIFO costs, it justifies treating a popped location as settled in this search implementation.

TIME-DEPENDENT ROAD COST

The function edge_travel_seconds does not apply one multiplier to an entire road merely because of its entry time. It tracks remaining free-flow work, advances to the next daily boundary, and continues at the new rate. At 09:59 a two-minute road under 2× peak congestion takes 60 seconds of peak time for 30 seconds of work, then 90 seconds off peak: 150 seconds total.

WHAT THE MODEL DOES NOT ESTIMATE

No live traffic, stop geolocation, boarding time, passenger waiting, railway train schedules, traffic signal cycles, or real operational restrictions are included. The animation shows the route computed from these assumptions; it is not observed vehicle movement.

A* and Dijkstra

The decision mechanism and its comparison baseline

A* DECISION RULE

A* stores candidate locations in a min-heap. The priority is **f(n)=g(n)+h(n)**: g is the best elapsed travel time from the start; h is the safe lower bound from n to the goal. It repeatedly pops the smallest f, skips outdated heap copies, and checks outgoing roads. A better candidate arrival replaces the saved g and parent link.

PSEUDOCODE

```
best_g[start] = 0; push(start, h(start))
while queue is not empty:
    n = pop_min_f(); skip if this entry is stale
    if n == goal: reconstruct parents; return route
    for each road n -> v:
        if road closed: continue
        candidate = best_g[n] + cost(road, depart + best_g[n])
        if candidate < best_g[v]:
            best_g[v] = candidate; parent[v] = n
            push(v, candidate + h(v))
return no route
```

The relevant code is **routing.py: _search**, especially heappop, the neighbor loop, the comparison against best_g, heappush, and parent reconstruction. The standard-library heap supplies an efficient queue; it does not perform route reasoning for us.

DIJKSTRA COMPARISON

Calling the same search engine with **h=0** gives Dijkstra's ordering by elapsed time alone. Both methods use the same graph, road-cost function, and closure set. Compare their minimum travel time, reachability, and number of expanded locations. The shared implementation makes this a useful **consistency check**, not an independent mathematical proof or a separately written oracle.

The heuristic need not save work on every trip. It helps when straight-line proximity predicts useful road progress; detours and congestion can weaken that signal. Runtime on a fourteen-node graph is too small and machine-sensitive to be the main claim, so the experiment emphasizes cost and expansion count.

Implementation

Data structures and inspectable evidence

KEY DESIGN CHOICES

Structure	Purpose
Adjacency lists	Available roads from each location
Min-heap	Select lowest f candidate
best_g map	Best known elapsed arrival per location
parent map	Reconstruct chosen route
expanded set	Ignore already settled locations
expansion log	Record neighbor decisions and pending queue

WHERE TO LOOK IN THE REPOSITORY

- **routing.py**: A*, Dijkstra mode, heuristic, time-dependent road cost, and optional reliability experiment.
- **sample_map.py**: seeded fictional graph and the two crossing edges.
- **review_demo.py**: code-first terminal run with trace, route costs, and no-route output.
- **app.py** and **web/**: input handling and visual explanation. Three.js only draws the result.
- **test_routing.py** and **experiments.py**: correctness checks and reproducible comparison data.

RECORDED DECISIONS

For each expanded location the trace stores g, h, f; each neighboring road's time and outcome (queued, closed, settled, or no improvement); and the valid waiting locations after that step. The browser exposes these values so a reviewer can choose a step and see why the next location was considered.

OPTIONAL RELIABILITY EXPERIMENT

A separate teaching experiment enumerates simple paths on this small graph and ranks an estimated P90 ETA under independent, approximately normal synthetic road delays. It is not part of A*'s objective or heuristic, and path enumeration is not scalable. Its delay spread is assumed, not measured.

Experiment design

Inputs, measures, and reproducibility

COMPARISON QUESTION

Does the admissible heuristic reduce the number of locations examined while keeping the same optimal modeled travel time as Dijkstra? Does changing departure time or blocking a tunnel change the selected route?

PROTOCOL

- Generate the same fixed-topology map at seeds 0, 1, 7, 17, and 42.
- Use departures 07:00, 08:30, 10:00, 12:00, 16:00, and 19:00.
- Test all four closure states and every ordered start/goal pair, including same-location and unreachable cases.
- Run A* and Dijkstra on identical graph/input data; record reachability, travel cost, and locations expanded.
- Use seed 17, A to D, for human-readable route-switch examples.

The Cartesian product is **5 × 6 × 4 × 14 × 14 = 23,520** cases. The experiment code raises an error if the two methods disagree on reachability or travel time. The separate unit suite covers 43,120 cases using eleven times, including just-before and just-after peak boundaries.

MEASURES AND INTERPRETATION

Measure	Meaning
Route time	Sum of arrival-dependent road costs
Reachability	Whether an open-road path reaches the goal
Nodes expanded	Valid queue pops whose roads were considered
Chosen tunnel	Whether the route crosses west or east

The aggregate averages use only reachable cases because the returned RouteResult carries an expansion count only on success. Unreachable cases are counted separately. Search expansion counts are not vehicle stops, and the animation's elapsed playback time is not the modeled trip time.

Results

Observed route switches and A* versus Dijkstra

CONCRETE SEED-17 CASES

Scenario	Crossing	Minutes	Expanded A* / D
12:00 open	west	8.30	11 / 14
08:30 open	east	12.46	11 / 13
12:00 west shut	east	9.17	10 / 11
12:00 east shut	west	8.30	10 / 12
12:00 both shut	none	—	—

The open midday case chooses the west tunnel. Morning congestion changes the best route to the east tunnel. Closing west also forces east; closing both makes this cross-rail trip unreachable. The difference is produced by the cost function and topology, not by animation rules.

AGGREGATE A* VERSUS DIJKSTRA

Cases	Reachable	Disagreements	A* mean expanded	D mean expanded
23,520	20,640	0	5.598	7.012

Among reachable cases, A* expanded fewer locations in 13,647 cases, the same number in 6,993, and more in 0. Its mean reduction was 20.2% relative to Dijkstra on this test set.

MEAN EXPANDED LOCATIONS (REACHABLE CASES)

Analysis and limits

What worked, what failed, and what would change

WHAT WORKED

- The search runs end to end in terminal and browser on user-selected inputs, including new seeds and closures.
- The default map makes the crossing constraint visible: changing time or closing west changes the route.
- A* and Dijkstra agree on all measured route costs and reachability states; A* expands fewer locations on many cases.
- The trace exposes every expansion, neighbor decision, and waiting queue rather than merely displaying a final route.

WHAT FAILED OR REMAINS LIMITED

- When both tunnel roads close, a cross-rail route cannot be found. This is a correct no-route result, not a program crash.
- The heuristic can be weak when road detours or congestion dominate straight-line distance; many cases show equal expansions.
- The map is generated and has no measured geographic or traffic accuracy. No real VIT ETA can be inferred.
- The topology remains fixed across seeds, so the experiment does not test arbitrary road-network structures.
- Dijkstra shares the search engine. Agreement detects some errors but cannot independently certify both implementations.
- The optional P90 estimate depends on synthetic, independent normal-delay assumptions; it is not observed reliability.

NEXT IMPROVEMENTS

Survey legal shuttle roads and stops, gather repeated travel times by period, and validate predicted times against held-out trips. Add passenger waiting and service frequency only after defining a new objective and state. Add an independent shortest-path oracle on tiny graphs to strengthen verification. For larger maps, replace exhaustive reliability path enumeration with a scalable method.

INTERPRETATION

The experiment supports a narrow conclusion: in this declared synthetic model, an admissible straight-line heuristic preserves the minimum modeled travel time and often reduces search expansions. It does not establish real-world route quality, bus efficiency, or a universal A* speedup.

Reproduce and disclose

Run instructions, references, and viva responsibility

EXACT RUN COMMANDS

```
python app.py                # open http://127.0.0.1:8000
python review_demo.py         # code-first test case
python review_demo.py --time 08:30 --seed 42 --close west
python review_demo.py --close west --close east
python -m unittest -v       # correctness checks
python experiments.py         # regenerate comparison JSON
```

Python 3.10+ runs the AI core without external packages. The local browser animation uses bundled Three.js. Rebuilding the PDFs requires ReportLab; it does not participate in route decisions. The repository README gives the full setup and optional browser check.

AI-TOOL DECLARATION AND ACADEMIC RESPONSIBILITY

An AI assistant helped draft and refine synthetic data, Python code, tests, interface, experiments, and this report. No external AI model or API makes route decisions at runtime. The submitted search executes in local inspectable Python. The team must confirm that its AI-assisted workflow is allowed by the course, verify the generated material, and state actual individual contributions truthfully. Each member must be able to explain and modify the AI core in the viva.

REFERENCES

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107, 1968. DOI: 10.1109/TSSC.1968.300136.

[2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, 1, 269–271, 1959. DOI: 10.1007/BF01386390.

[3] Course-project guidelines email supplied by the instructor; requirements for self-implemented AI core, search formulation, comparison, report, slides, live demo, and individual viva.

VIVA-READY EXPLANATION

“The state is a location and its arrival time. For each open road, we calculate its travel time when the shuttle reaches it. A* expands the waiting location with the lowest elapsed time plus a safe straight-line estimate. The best-arrival and parent maps record improvements. Dijkstra sets that estimate to zero for comparison. Our result is a modeled optimum on synthetic data.”